

Exp 2: READ AND WRITE THE REGISTER VALUES OF A MICROCONTROLLER

**Name: Pradip Maity
CDSID: PMAITY**



Table of Contents

S.No.		Page No
1	Introduction	3
2	Scope	3
3	Prerequisite	3
4	Connection Diagram	3
5	Working Principle	4
6	Application Code	4
7	Output	5
8	Hardware Image	5
9	Appendix	6

1. Introduction:

This project demonstrates how to read and write register values of a microcontroller using the Raspberry Pi Pico. A register is a small memory location inside the microcontroller used to store data or control hardware functions. Each register is associated with a unique memory address.

A menu-driven program is implemented using UART serial communication, allowing register access through a serial console. This approach provides insight into low-level programming and direct hardware interaction in embedded systems.

2. Scope:

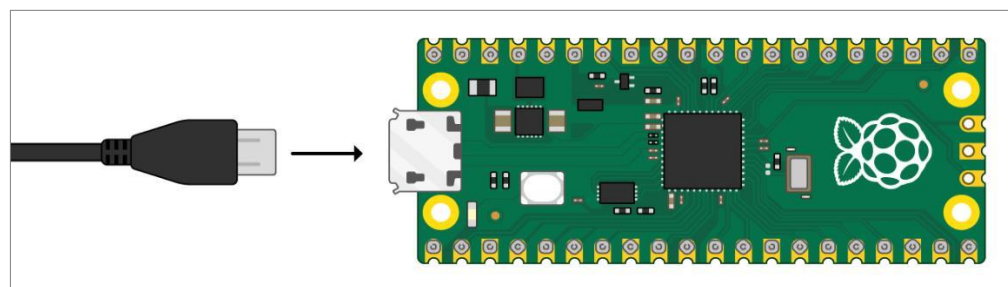
This project provides an understanding of direct register access using memory addresses. It introduces low-level control of microcontroller hardware and demonstrates how serial communication can be used for user interaction. The concepts learned are useful in embedded system development, debugging, and hardware control applications.

3. Prerequisite:

Hardware: Raspberry Pi Pico Microcontroller, USB Connector, Breadboard

Software: VS Code (with Pico SDK), PuTTY Serial Console (in Windows)

4. Connection Diagram:



5. Working Principle:

UART serial communication is used to interact with the user through the PuTTY console. After initialization, a menu is displayed with options to read or write register values.

For the read operation, a register address is entered in hexadecimal format. This address is converted into a pointer, and the value stored at that location is accessed and displayed on the console.

For the write operation, both the register address and the value are provided. The given address is converted into a pointer, and the specified value is written directly to that memory location.

The process uses direct memory access through pointers, enabling interaction with hardware registers. The menu continues to operate until an exit option is selected.

6. Application code:

```
#include <stdio.h>
#include "pico/stdlib.h"

void flush_input() {
    while (getchar_timeout_us(0) != PICO_ERROR_TIMEOUT);
}

int get_choice() {
    int ch;

    while (true) {
        ch = getchar();
        if (ch == '\r' || ch == '\n') continue;
        if (ch >= '1' && ch <= '3') {
            printf("%c\n", ch);
            return ch - '0';
        } else {
            printf("\nInvalid input! Enter 1, 2, or 3\n");
        }
    }
}

uint32_t get_hex_input() {
    char buffer[20];
    int index = 0;
    int ch;

    while (true) {
        ch = getchar();
        if (ch == '\r' || ch == '\n') {
            buffer[index] = '\0';
            break;
        }
        if (index < sizeof(buffer) - 1) {
            buffer[index++] = ch;
            putchar(ch);
        }
    }
    printf("\n");
    uint32_t value = 0;
    sscanf(buffer, "%x", &value);
    flush_input();
    return value;
}
```

```
void read_register() {
    printf("\nEnter register address (in hex): ");
    uint32_t address_val = get_hex_input();
    volatile uint32_t *address = (volatile uint32_t *)address_val;
    uint32_t value = *address;
    printf("\nRegister value at 0x%08X: 0x%08X\n", address_val, value);
}

void write_register() {
    printf("\nEnter register address (in hex): ");
    uint32_t address_val = get_hex_input();
    volatile uint32_t *address = (volatile uint32_t *)address_val;
    printf("\nEnter value to modify (in hex): ");
    uint32_t value = get_hex_input();
    uint32_t original = *address;
    uint32_t modified = value;
    *address = modified;
    sleep_ms(100);
}

int main() {
    stdio_init_all();
    while (!stdio_usb_connected()) {
        sleep_ms(100);
    }
    printf("\n=== Register Read/Write Menu ===\n");
    while (true) {
        printf("\n1. Read register\n");
        printf("\n2. Write register\n");
        printf("\n3. Exit\n");
        printf("\nEnter your choice: ");
        int choice = get_choice();
        flush_input();
        switch (choice) {
            case 1:
                read_register();
                break;
            case 2:
                write_register();
                break;
            case 3:
                printf("\nExiting program...\n");
                fflush(stdout); // force send
                sleep_ms(500);
                return 0;
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}
```

7. Output:

```
COM4 - PuTTY

=== Register Read/Write Menu ===

1. Read register
2. Write register
3. Exit
Enter your choice: 1

Enter register address (in hex): 20000000

Register value at 0x20000000: 0x20042000

1. Read register
2. Write register
3. Exit
Enter your choice: 2

Enter register address (in hex): 20000000

Enter value to modify (in hex): 12345678

1. Read register
2. Write register
3. Exit
Enter your choice: 1

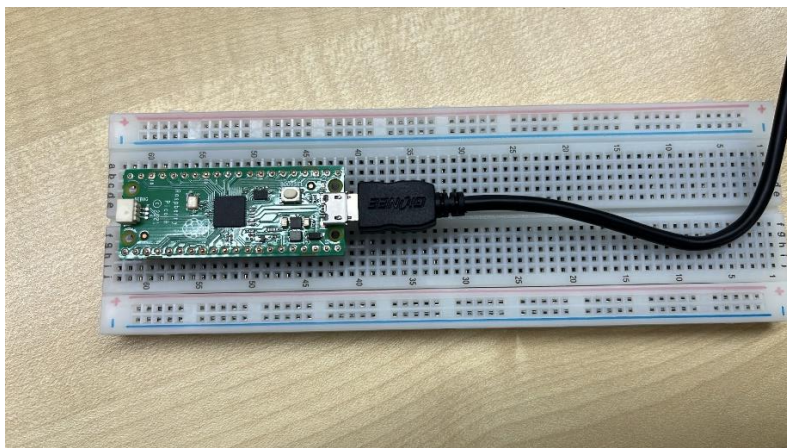
Enter register address (in hex): 20000000

Register value at 0x20000000: 0x12345678

1. Read register
2. Write register
3. Exit
Enter your choice: 3

Exiting program...
█
```

8. Hardware Image:



9. Appendix:

References

- Official Documentation for Raspberry Pi Pico and Pico W
- Technical Datasheet for Raspberry Pi Pico
- GitHub Repository for Raspberry Pi Pico SDK
- PuTTY Serial Console Documentation